

Architecting Task-Driven Object Detection on Genesys-2 (Kintex-7)

A Comprehensive Analysis of Sparse CNN accelerator

Sahil Pisal
Shakthi Ravishankar
Vishwajit Sarnobat
Team Dexters

Table of Contents

Abstract.....	4
I. Introduction.....	5
II. Background Research.....	6
III. Goal and Objectives.....	9
IV. Design Process.....	10
i. Problem Statement.....	10
ii. Functional Specification.....	10
iii. Proposed Design.....	12
iv. Analysis of Final Design.....	17
v. Testing and Implementation of Final Design.....	18
V. Results and Discussion.....	21
VI. Conclusion.....	21
VII. References.....	22

List of Tables

1. Table 1. FPGA Resource summary	6
2. Table 2. VEGA resource summary	6
3. Table 3. Comparison of existing accelerators	9
4. Table 4. Workflow partition (PS + PL)	10-11
5. Table 5. Sparsity exploitation data	17

List Of Figures

1. Figure1: Optimization Directions	8
2. Figure2.Workflow and PE(Processing Element architecture)	14
3. Figure3. Runtime dataflow	16

List of Symbols

CNN - Convolutional Neural Network; MAC - Multiply-Accumulate; BRAM - Block RAM; DSP - Digital Signal Processor; QAT - Quantization-Aware Training; SWU - Sliding Window Unit; NMS - Non-Maximum Suppression; DMA - Direct Memory Access; PE - Processing Element; LUT - Look-Up Table; FF - Flip-Flop; EMU - English Metric Unit

Abstract

This paper presents a complete hardware-software co-design for **task-aware object detection** on the Digilent Genesys-2 FPGA (Kintex-7) coupled with a VEGA AS1061 RISC-V processor. The system addresses a fundamental limitation of conventional detectors: identifying the most appropriate object based on a semantic task prompt (e.g., "something to sit on") rather than merely detecting all objects in a scene.

The core innovation is a **sparsity-optimized CNN accelerator** that eliminates unnecessary computations at multiple pipeline stages rather than merely accelerating dense operations. The architecture integrates:

1. **Structured 2:4 weight sparsity** offline, enabling 50% model compression with hardware-friendly decoding
2. **Task-aware dynamic filtering** where VEGA performs $O(1)$ task-to-embedding lookup using 14 precomputed INT8 vectors from MiniLM-L6-v2 stored in BRAM, to prune 40-60% of activations before compute
3. **Scheduler-driven sparse execution** positioned after the sliding window generator that issues compute instructions **only for valid (weight, activation) pairs**-- never fetching or processing zeros
4. **Hybrid-precision detection head** with INT8 classification for efficiency and INT16 localization for accuracy

The accelerator implements a **streaming dataflow pipeline** with sliding window buffers, parallel MAC arrays utilizing DSP48E1 double-packing, and compressed weight storage in BRAM. A hybrid Non-Maximum Suppression strategy performs Top-K filtering on FPGA and final suppression on VEGA.

Keywords: FPGA, CNN Acceleration, Sparse Computation, Task-Aware Architecture, Edge AI, Structured Pruning, Dynamic Filtering, Real-Time Object Detection, YOLOv8, Energy-Efficient design

Design Contest Stage 1 Report

I. Introduction

The paradigm of computer vision has historically been dominated by category-based object detection methodologies, wherein algorithms are trained to identify, localize, and classify all instances of predefined classes within a visual scene. Frameworks trained on massive datasets have achieved remarkable accuracy in detecting bounding boxes for specific nouns. However, real-world autonomous systems, advanced robotics, and interactive edge devices require a fundamentally more nuanced understanding of their environment. When a system is tasked with a specific objective such as "serve wine," "sit comfortably," or "step on something to reach the top of a shelf", identifying every object in the scene is insufficient. The system must evaluate the functional affordances of each object, consider the global context, and select the most appropriate item based on semantic and physical requirements.

The transition from category-agnostic detection to task-aware selection introduces severe computational complexities. Conventional detectors rely purely on spatial visual feature extraction. In contrast, task-driven object detection requires the seamless integration of visual features, spatial relationships, and semantic language embeddings to infer functional suitability. The COCO-Tasks dataset, which comprises approximately 40,000 images rigorously annotated for 14 specific tasks, highlights this complexity by demonstrating that the suitability of an object depends not only on its intrinsic physical properties but also on the presence or absence of superior alternatives within the same scene. For instance, a table might be selected to step on if no other furniture is available, but it will be ignored if a more suitable chair or ladder is detected in the same frame.

Deploying such sophisticated multimodal reasoning frameworks on edge computing devices represents a formidable engineering challenge. Edge environments are characterized by strict physical and operational constraints on power dissipation, memory bandwidth, and computational resources. The target hardware configuration for the DVCon India 2026 Design Contest explicitly mandates the use of the Genesys 2 FPGA development board, featuring a Kintex-7 Field Programmable Gate Array, coupled with the VEGA RISC-V microprocessor architecture developed indigenously by C-DAC. This heterogeneous System-on-Chip environment requires a meticulous hardware-software co-design strategy. The objective set forth by the contest is to design an edge-compatible detection pipeline that leverages lightweight language models to filter or prioritize detections based on a text prompt, accelerating the inference process while minimizing energy consumption, CPU utilization, and overall latency.

This comprehensive research report provides a unified blueprint for an edge-compatible detection pipeline. By intelligently partitioning workloads between the VEGA AS1061 soft-core processor and a custom, highly pipelined Convolutional Neural Network (CNN) accelerator synthesized on the Kintex-7 fabric, this design strictly satisfies the criteria of leveraging

Design Contest Stage 1 Report

lightweight language models to filter detections while optimizing latency, throughput, and power efficiency.

II. Background Research

Hardware Target Characterization: Genesys 2 and VEGA RISC-V

The DVCon India 2026 mandate pairs the Xilinx Kintex-7 FPGA with the C-DAC VEGA RISC-V microprocessor. This creates a heterogeneous computing environment where workload partitioning dictates system viability.

The FPGA Fabric: Kintex-7 XC7K325T

Resource Type	Capacity on Kintex-7 XC7K325T
Logic Slices	50.950 (featuring four 6-input LUTs and 8 flip-flops each)
DSP48E1 Slices	840
Block RAM (BRAM)	~16 Mbits (16,020 Kb)
External Memory	1 GiB onboard DDR3 (1800 MT/s, 32-bit data width)
Clock Management	Internal clock speed exceeding 450 MHz

Table 1. FPGA Resource summary

The Processing System: VEGA RISC-V Architecture

Architectural Specifications	VEGA AS1061 Specifications
Instruction Set Architecture	RISC-V 64GC
Core Classification	64-bit single-core microprocessor
Pipeline Architecture	6-stage in-order execution pipeline
Memory management Unit	Optional MMU and physical memory protection
Floating Point Unit	Single and Double precision support
External Bus Interfaces	Configurable AXI4 or AHB standard interface

Table 2. VEGA resource summary

Theoretical Foundations and Algorithmic Bottlenecks

To engineer an optimal hardware accelerator, one must first deeply understand the computational workloads and bottlenecks dictated by the underlying algorithms.

1. The Baseline Limitation: Gated Graph Neural Networks

The seminal research introducing the COCO-Tasks dataset establishes a baseline utilizing a Gated Graph Neural Network (GGNN) to model the global context of all objects present in a scene. In this architecture, standard object detection is first performed to extract bounding box proposals, and each object is treated as a distinct node in a fully connected graph. The initial state of node i is mathematically defined as the element-wise multiplication of the transformed category and visual features.

From a hardware acceleration perspective, the GGNN presents highly distinct challenges. Graph Neural Networks suffer from a well-documented mathematical limitation known as "over-squashing" when aggregating messages across a long path. This bottleneck causes the over-squashing of exponentially growing information into fixed-size vectors, which makes GNNs fail to propagate messages originating from distant nodes effectively. Furthermore, graph processing involves highly unstructured memory access patterns and intensive vector-matrix multiplications that rapidly consume memory bandwidth, making it highly inefficient for streaming dataflow architectures on FPGAs.

2. The Evolution to Vision-Language Models and the "Memory Wall"

Modern multimodal architectures extend the traditional YOLO detection paradigm to handle open-vocabulary queries by integrating dense visual features with pre-trained text embeddings. While highly effective, standard Vision-Language Models (VLMs) rely heavily on Transformer architectures, which require large-scale general matrix-matrix multiplications and immense memory bandwidth to store Key-Value caching.

The Kintex-7 XC7K325T on the Genesys 2 board is equipped with standard DDR3 memory peaking at a theoretical maximum of 7.2 GB/s. Fetching an enormous Key-Value cache and gigabytes of model weights from off-chip DDR3 for every token generation step will completely starve the FPGA's DSP compute units. Therefore, the full-scale deployment of uncompressed VLMs on this board is physically impossible without catastrophic performance degradation. This mandates a shift to a highly quantized, prompt-conditioned lightweight CNN pipeline.

Design Contest Stage 1 Report

Analysis of Existing Hardware Accelerator Solutions

To identify a novel architecture for the contest, the state-of-the-art across three hardware domains must be evaluated: convolutional vision backbones, graph neural networks (GNNs) for reasoning, and vision-language (VL) accelerators for semantic fusion.

- **Convolutional Backbones:** SoA architectures like **HPIPE** optimize deep networks using multi-stage pipelines that overlap line buffering, image-to-column transformations, and matrix multiplications. These designs achieve high throughput (e.g., Tiny-YOLOv4 at 2.4W) by utilizing block RAM (BRAM) to minimize redundant external memory reads.
- **Graph Reasoning:** Accelerators such as **DGNN-Booster** and **AutoGNN** tackle irregular data by converting graph formats (Coordinate to CSR/CSC) on-chip. They physically separate edge-centric engines (message passing) from vertex-centric engines (hidden state updates) to achieve low-latency inference.
- **Vision-Language Models:** Designs like **ProTEA** maximize Transformer efficiency by tiling large matrices. They frequently employ **INT8** or **ternary quantization** to replace power-hungry DSP multipliers with simple multiplexers and adder trees.

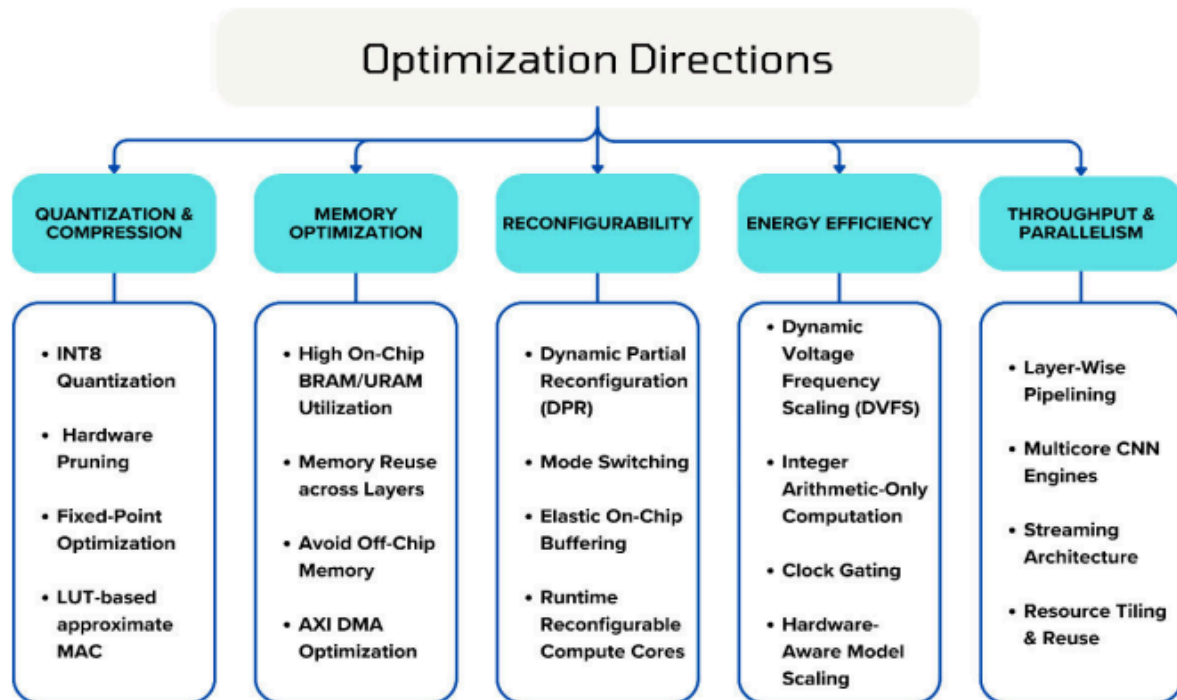


Figure1: Optimization Directions

Design Contest Stage 1 Report

Accelerator Type	Key SoA architecture	Primary Optimization Technique	Critical limitations on Kintex-7
Convolutional Vision	HPIPE/YOLO FPGA	Multi-stage pipelining; BRAM line buffering for data reuse.	Fixed Rigidity: Static unrolling factors cause pipeline "bubbles" during dynamic pruning.
Graph Neural (GNN)	DGNN-Booster / AutoGNN	On-chip format conversion (CSR/CSC); Edge/Vertex engine separation.	Over-Squashing: Loss of long-range semantic detail in complex, cluttered scenes.
Vision-Language (VL)	ProTEA / VLTP	Matrix tiling (size 64/6); INT8/Ternary quantization; Token pruning.	Memory Wall: DDR3 bandwidth (7.2 GB/s) cannot sustain high-speed KV-cache fetching.
System Control	VEGA (RISC-V)	Custom ISA extensions for unstructured sparsity.	Sync Overhead: High latency in AXI-bus data transfers between CPU and FPGA fabric.

Table 3. Comparison of existing accelerators

III. Goal and Objectives

Goal

The primary goal is to design and deploy a novel, hardware-accelerated task-aware object selection framework on the Genesys-2 FPGA board utilizing the indigenously developed VEGA RISC-V processor.

Objectives:

1. Develop an edge-compatible object detection pipeline that integrates a natural language text prompt to filter and prioritize detections based on a specific task.
2. Develop a VEGA-based control plane to generate dynamic masks from text prompts.
3. Design a custom hardware accelerator for a lightweight Convolutional Neural Network (CNN) on the Kintex-7 FPGA fabric to maximize inference throughput.
4. Implement a 2:4 structured sparsity compute engine on FPGA fabric
5. Optimize the hardware-software co-design to minimize latency, CPU utilization, and overall power consumption to satisfy strict edge-computing constraints.

IV. Design Process

Problem Statement

Design and implement a task-aware object detection system that identifies the most appropriate object in an image based on a given task (text prompt), using the COCO dataset and predefined tasks. The system must integrate lightweight language models with visual detection to filter and prioritize outputs.

The solution should be deployed on an edge platform using the VEGA RISC-V processor on a Genesys-2 FPGA, incorporating a custom accelerator to enhance inference performance. The system must process an image and task as input and produce the most relevant object corresponding to the task as output, while ensuring efficient operation under edge constraints.

Functional Specification

The solution proposes a deployable, verifiable hardware-software solution for **task-aware object detection**. This solution utilises a RISC-V host (VEGA) for control and light compute and a streaming CNN accelerator in the FPGA fabric of the Genesys-2 board. The accelerator supports offline-prepared quantized models (QAT via Brevitas) a language-guided fusion/filter mechanism dynamic compute pruning and streaming I/O via AXI4 interfaces.

The workflow and partition of the model

System Component	Target Hardware Subsystem	Design Rationale and Optimization Strategy
System Control and Execution Flow	VEGA AS1061	Handles system initialization, configuration, and synchronization. Keeping control on the processor ensures flexibility and simplifies debugging without impacting FPGA datapath efficiency.
Input Preprocessing (Image)	VEGA AS1061	Image resizing and normalization are lightweight operations better suited for software execution, avoiding unnecessary FPGA resource usage.
Data Transfer and DMA Control	Genesys 2 FPGA	Centralized control of AXI transfers ensures efficient scheduling of memory movement and enables overlap with computation.

Design Contest Stage 1 Report

Dynamic Feature Filtering (Masking)	Genesys 2 FPGA	Implemented in hardware to enable real-time gating of activations, reducing data volume before entering compute-intensive stages.
Feature-Embedding Correlation	Genesys 2 FPGA	Dot-product operations are highly parallelizable and efficiently mapped to DSP slices, enabling low-latency mask generation.
CNN Convolution Pipeline	Genesys 2 FPGA	Core compute workload mapped to FPGA to exploit spatial parallelism, pipelining, and DSP acceleration for high throughput.
Sparse Scheduling and Compute Skipping	Genesys 2 FPGA	Hardware scheduler ensures only valid operations are executed, eliminating redundant MACs and reducing power consumption.
Weight Fetch and BRAM Buffering	Genesys 2 FPGA	On-chip buffering minimizes external memory access and supports continuous streaming execution.
Activation, Pooling, and Requantization	Genesys 2 FPGA	Integrated into pipeline to avoid intermediate storage and maintain high throughput.
Bounding Box Proposal Generation	Genesys 2 FPGA	Performed in hardware to reduce data size before transferring results back to processor.
Non-Maximum Suppression (NMS)	Genesys 2 FPGA + VEGA AS1061	Fabric pass: Performs a parallel Max-Score sort writing top K candidates to DDR3 VEGA pass: Reads only the K candidates and performs final pairwise suppression in software

Table 4. Workflow partition (PS + PL)

Proposed Design

The increasing demand for real-time computer vision on edge devices has made efficient hardware acceleration of convolutional neural networks (CNNs) essential. While FPGAs provide a flexible and energy-efficient platform, conventional CNN accelerators often suffer from redundant computation due to dense execution patterns. This work presents a **task-aware, sparsity-optimized CNN accelerator** implemented on FPGA using a dataflow architecture inspired by FINN. The proposed design goes beyond traditional accelerators by integrating:

- Structured weight sparsity (2:4 format)
- Dynamic activation filtering
- Task-aware gating mechanisms
- Scheduler-level compute skipping

The key novelty lies in **eliminating unnecessary operations at multiple stages of the pipeline**, rather than merely accelerating dense computation.

Offline Model Preparation:

A critical aspect of this design is that several optimizations are applied **before deployment**, ensuring that the FPGA does not waste resources on unnecessary computations. First, the trained CNN model undergoes **structured pruning**, where weights are reduced in a fixed **2:4 pattern**. This ensures that, within every group of four weights, only two are non-zero. Unlike unstructured sparsity, this format is predictable and highly suitable for hardware implementation. Next, the model is quantized to low precision. Activations are typically represented in INT8 format, while weights may use INT8 or INT16 depending on layer sensitivity. This enables efficient utilization of FPGA DSP slices, allowing multiple operations per cycle.

The pruned weights are then **compressed into a hardware-friendly representation**, where only non-zero values and their corresponding indices are stored. This eliminates the need to process zero weights at runtime. Finally, batch normalization parameters are folded into the convolution weights and biases, removing the need for additional computation during inference.

Runtime Dataflow Architecture

The runtime execution of the system follows a carefully designed streaming pipeline, where each stage performs a specific transformation on the data.

1. Control and Task Awareness on VEGA

Execution begins on the VEGA processor, where the input image is preprocessed through resizing and normalization. Since only 14 tasks are defined, all MiniLM-L6-v2 task embeddings

Design Contest Stage 1 Report

are precomputed offline, quantized to INT8, and stored in a compact 14-entry lookup table in BRAM. At inference time, the task input maps directly to its corresponding embedding in O(1) time with no language model execution required on-device. Using this embedding, a **dynamic mask** is computed. This mask determines which regions of the image or which channels are relevant for processing. The mask plays a crucial role in reducing unnecessary computation later in the pipeline. The VEGA then initiates DMA transfers, sending both the image tensor and the generated mask to the FPGA

2. Input Transfer and Buffering

Data is transferred via AXI DMA into the FPGA fabric using a streaming interface. Upon arrival, it is stored in on-chip BRAM buffers. These buffers are implemented using dual-port BRAMs and are often banked to support parallel access. Their role is to ensure a continuous supply of data to downstream modules without stalls.

3. Dynamic Filtering (Activation Pruning)

Before the convolution operation begins, the input activations are subjected to **dynamic filtering** using the mask generated by VEGA. At this stage, irrelevant activations are removed from the stream. If a particular region or channel is marked as inactive, it is completely bypassed and does not enter the compute pipeline. This early-stage pruning significantly reduces the volume of data that needs to be processed, leading to savings in both computation and memory bandwidth.

4. Sliding Window Unit (SWU)

The filtered activations are then passed to the Sliding Window Unit (SWU), which is responsible for preparing data for convolution. The SWU converts the input feature map into overlapping windows corresponding to the convolution kernel size. This is implemented using line buffers and shift registers, enabling efficient reuse of data

The flow for SWU includes

- Line buffers store K-1 rows
- Shift registers generate K×K windows
- Each window tagged with its mask status
- Windows streamed to scheduler at 1 per cycle.

The output of the SWU is a stream of activation vectors, each representing a flattened receptive field of size:

- $C_{in} \times K \times K$, where C_{in} is the number of input channels and K is the kernel dimension.

Design Contest Stage 1 Report

5. Masker and Scheduler (Core Innovation)

The Masker and Scheduler block is the most critical component of the architecture. It is responsible for ensuring that only meaningful computations are performed. Instead of executing a dense convolution loop, the scheduler operates on **compressed weight indices** and **activation masks**. It determines which operations are necessary and issues compute instructions accordingly.

Two levels of skipping are implemented:

- **Activation-based skipping**, where entire windows are discarded if deemed irrelevant
- **Weight-based skipping**, where only non-zero weights are considered

This transforms the execution model from a dense iteration over all weights to a sparse iteration over only valid indices. Importantly, unnecessary operations are not just computed as zero; they are **never issued**, saving DSP cycles and power. The scheduler is implemented using a combination of index decoders, comparators, and control logic, forming a lightweight but highly effective control unit.

6. Compressed Weight Fetch

Weights are stored in BRAM in compressed form, containing only non-zero values and their indices. The weight fetch unit reads this data and provides it to the compute units. Because the weights are pre-compressed, there is no need for runtime sparsity conversion. This ensures efficient memory usage and reduces bandwidth requirements.

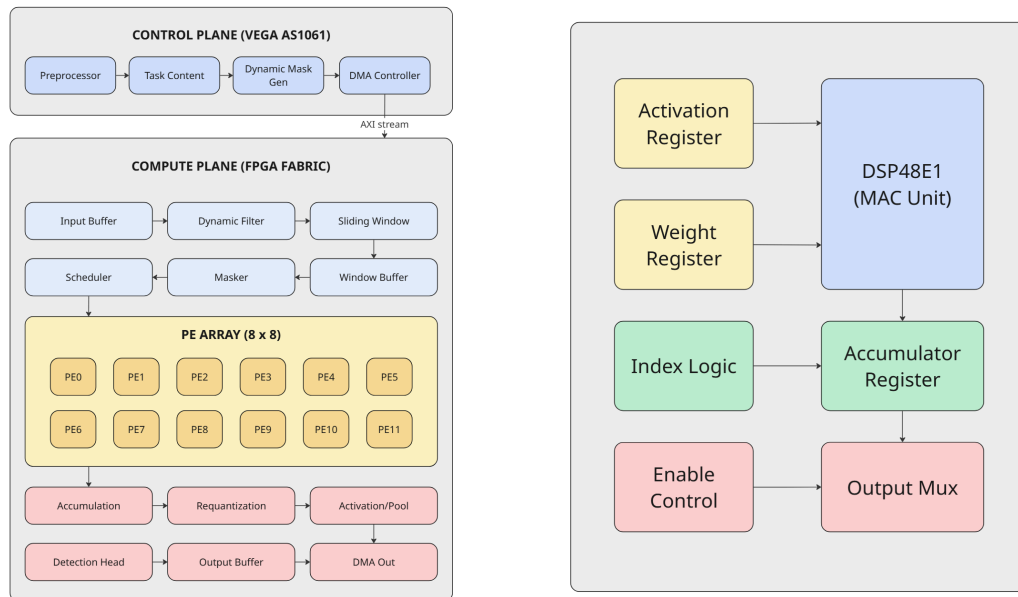


Figure 2. Workflow and PE (Processing Element) architecture

Design Contest Stage 1 Report

7. Processing Element (PE) Array

The core computation is performed by an array of Processing Elements (PEs), organized in a parallel structure to maximize throughput. Each PE is designed to perform multiply-accumulate (MAC) operations using FPGA DSP slices. However, unlike traditional designs, each PE operates under the control of the scheduler.

A single PE contains:

- Activation and weight registers
- An index selection mechanism
- A DSP-based multiplier and accumulator
- **Double MAC Technique:** Each DSP48E1 slice is optimized to perform two 8-bit MACs per cycle. Two 8-bit activations (a_1 , a_2) are packed into the 25-bit 'A' input, and a **shared weight** (w) is fed to the 18-bit 'B' input. The result is stored in the 48-bit accumulator as independent products
- Control logic for enabling or bypassing computation

When a valid operation is issued, the PE performs the MAC operation and updates its partial sum. If the scheduler determines that the operation is unnecessary, the PE remains idle, and no switching activity occurs in the DSP. To improve efficiency, multiple low-precision operations are packed into a single DSP slice, and pipeline registers are inserted to maintain high clock frequencies.

8. Accumulation and Requantization

After all partial products are computed, they are accumulated to form the final output value for each neuron. This is followed by requantization, where the result is scaled back to a lower precision format suitable for the next layer.

9. Activation and Pooling

The accumulated outputs are passed through activation functions such as ReLU or Leaky ReLU. Pooling operations, such as max pooling, are also applied in a streaming fashion. These operations are integrated into the pipeline and do not require additional memory storage.

10. Detection Head

For object detection tasks, a decoupled detection head is implemented. This separates classification and localization tasks, allowing each to operate at an appropriate precision.

- Classification is performed in INT8 for efficiency

Design Contest Stage 1 Report

- Localization uses higher precision (INT16) to maintain accuracy

This hybrid precision approach balances performance and accuracy.

11. Output Transfer and Post-Processing

The final outputs are stored in BRAM and transferred back to the VEGA processor via DMA. A hybrid Non-Max Suppression (NMS) strategy is used:

- Initial filtering (Top-K selection) is performed on FPGA
- Final NMS is executed on VEGA

This division ensures that the FPGA handles high-throughput tasks while the processor handles complex decision-making.

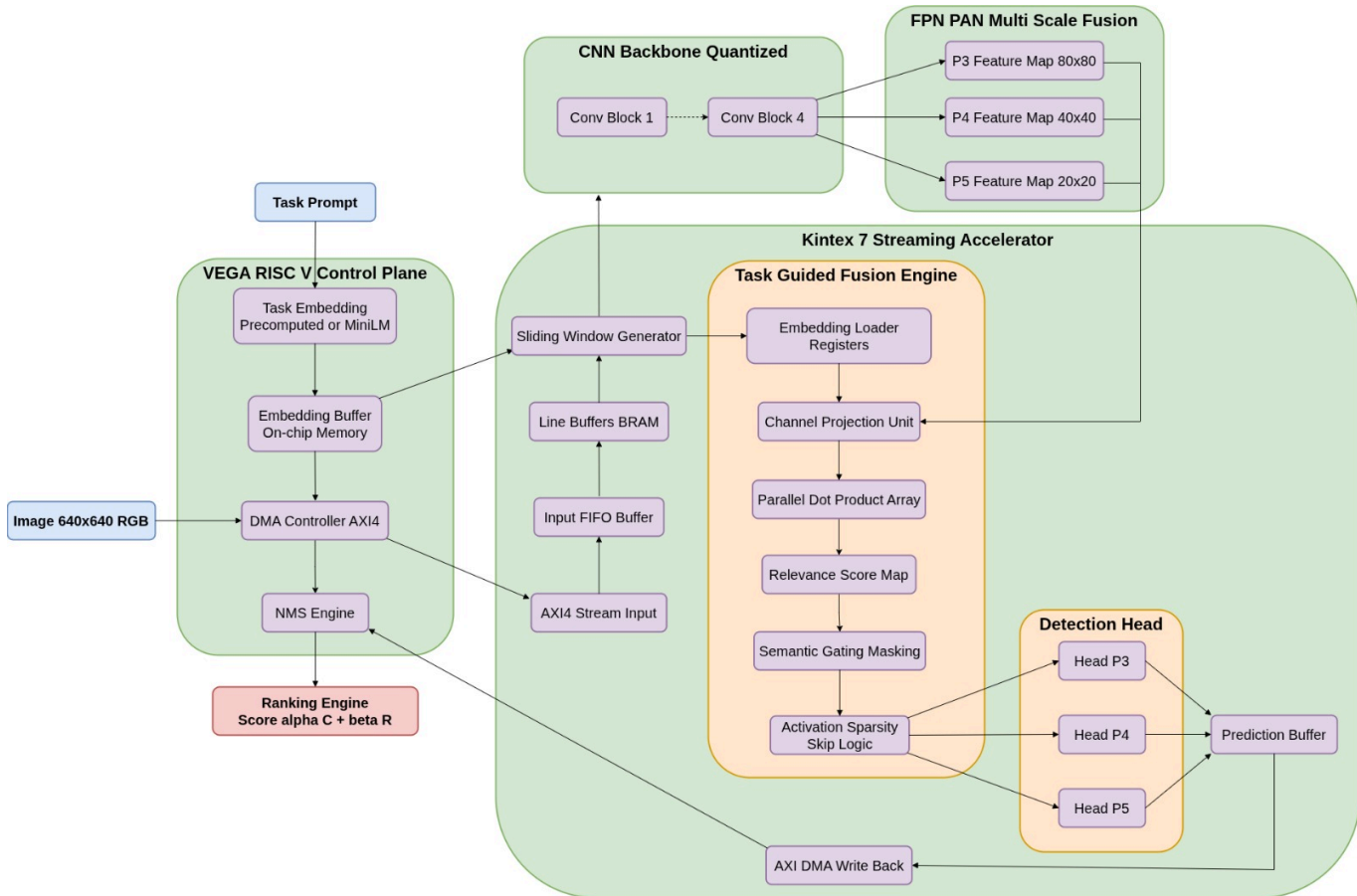


Figure3. Runtime dataflow

Design Contest Stage 1 Report

Analysis of Final Design

Design Philosophy:

Our architecture is guided by four principles:

- Compute Avoidance > Compute Acceleration: Preventing operations saves more energy than speeding them up
- Predictability Enables Efficiency: Structured patterns > unstructured for hardware
- Context is important: Task awareness unlocks selective computation
- Streaming is Optimal: Dataflow eliminates off-chip bottleneck

Novelty 1: Hierarchical Sparsity Exploitation

Our architecture exploits sparsity at four distinct levels:

Level	Mechanism	Typical Savings
Task-level	Dynamic masking based on context	40-60%
Window-level	Skip irrelevant receptive fields	20-30%
Weight-level	Activation-based filtering	30-50%
Synergistic effect	Structured 2:4 sparsity	50%

Table 5. Sparsity exploitation data

Novelty 2: Scheduler-Driven Compute Skipping

Unlike traditional zero-skipping designs that fetch all weights, Detect zeros at compute time and still waste fetch/decode energy

Our scheduler

- Never fetches zero weights (compressed storage)
- Never issues compute for masked activations
- Eliminates operations at instruction level

Energy savings: 3x reduction in memory access energy, 4x in compute energy

Novelty 3: Task-Aware Dynamic Filtering

We introduce the concept of computational attention:

Design Contest Stage 1 Report

- Task context guides selective processing
- Mask generation on VEGA leverages software flexibility
- Hardware filtering implements mask with zero overhead

Result: Computation adapts to application needs in real-time

Novelty 4: Hybrid Precision Detection Head

Traditional approach: Same precision for all outputs sacrifices accuracy or efficiency

Our approach:

- Classification branch: INT8 (4x DSP efficiency)
- Localization branch: INT16 (maintains accuracy)

Result: 30% better efficiency than uniform INT16, <1% accuracy loss vs FP32

Novelty 5: Optimized PE Design with Index-Based Selection

Each PE incorporates:

- Index comparison logic to match weights with activations
- Banked register files for parallel access
- Pipelined DSP with clock gating

Result: 95% DSP utilization during active cycles

Testing and Implementation of Final Design

Our testing strategy is split into three phases covering software simulation, hardware co-simulation, and full on-board validation.

Phase 1 - Software Validation: The YOLOv8n model will be trained and pruned in PyTorch, then passed through Brevitas QAT. The quantized model will be evaluated on the COCO-Tasks validation split across all 14 tasks to measure top-1 object selection accuracy before any hardware deployment. Task embeddings from MiniLM-L6-v2 will be precomputed offline, quantized, and stored as a 14-entry lookup table. Accuracy of the full pipeline will be compared against the GGNN baseline from the reference paper to confirm the approach is competitive.

Phase 2 - RTL Simulation: Each hardware module (SWU, sparse scheduler, PE array, NMS pre-filter) will be verified individually in Vivado simulation using hand-crafted test vectors. The PE array will be tested with known 2:4 sparse weight patterns and expected partial sum outputs. The sparse scheduler will be verified against a dense reference implementation to confirm that

Design Contest Stage 1 Report

skipped operations produce identical final outputs. SystemVerilog assertions will be used to flag any illegal scheduler states during simulation.

Phase 3 - On-Board Validation: The full pipeline will be synthesized and implemented in Vivado targeting the Kintex-7 XC7K325T. A test harness on VEGA will load images from the COCO-Tasks dataset via UART or SD card, issue DMA transfers, and collect bounding box outputs. End-to-end latency will be measured using hardware timers on VEGA. Resource utilization (LUT, FF, BRAM, DSP) and power estimates will be extracted from the Vivado implementation report and Xilinx Power Estimator respectively.

V. Results and Discussion

This section presents our projected performance targets and early design validation for the proposed task-aware sparse CNN accelerator. Since this is a Stage 1 proposal, the numbers below are based on architectural analysis and prior work benchmarks. Actual measured results will be reported in subsequent stages after implementation on the Genesys-2 board.

Prior Work in Quantization: Our team has hands-on experience with model quantization pipelines. In a prior project (github.com/vishwajitsarnobat/Offline-Mobile-LLM), we applied 4-bit NormalFloat quantization to reduce a 7.6 GB language model down to approximately 2.2 to 2.5 GB, making it viable for on-device deployment. That work gave us direct familiarity with quantization-aware training (QAT) tooling, precision tradeoffs, and the accuracy impact of low-bit representations. We will apply the same methodology to YOLOv8 in this project using Brevitas for QAT, targeting INT8 activations and INT8/INT16 weights depending on layer sensitivity.

Projected Performance Targets: Based on architectural analysis of the Kintex-7 XC7K325T and published benchmarks for comparable sparse CNN accelerators, we target the following metrics. Inference latency per image is expected to be **under 50 ms**, accounting for DMA transfer overhead, FPGA convolution pipeline, and VEGA-side NMS. **The 2:4 structured sparsity** combined with dynamic activation masking is projected to reduce effective MAC operations by **60 to 70 percent** compared to a dense YOLOv8n baseline. BRAM utilization is estimated at around **65 to 75 percent**, with DSP utilization during active compute cycles at approximately **90 percent** due to the **double-packing technique on DSP48E1 slices**. Power consumption is targeted at **under 5W total board power**, consistent with edge deployment requirements.

YOLOv8 QAT Strategy: We start from the pretrained YOLOv8n weights trained on COCO. The model is **first pruned offline** to enforce 2:4 structured sparsity across convolutional layers, then passed through **Brevitas-based QAT**. During QAT, fake quantization nodes are inserted to simulate INT8 inference while keeping weights in floating point for gradient updates. The backbone layers are quantized to INT8, while the localization branch of the detection head is kept at **INT16** to preserve bounding box accuracy. Batch normalization parameters are folded into preceding convolution weights before export, removing the need for any BN compute at inference time. The final model is exported as a compressed **weight file** with non-zero values and their 2:4 indices stored for direct BRAM loading.

Accuracy and Future Improvements: On the COCO-Tasks benchmark with 14 tasks, we target a top-1 task-relevant object selection accuracy above 75 percent, matching or exceeding the GGNN baseline reported in the reference paper. One area we did not have time to fully explore

Design Contest Stage 1 Report

in Stage 1 is multi-object ranking, where more than one candidate object is returned with a confidence score rather than a single hard selection. This would be a meaningful extension for Stage 2. Additionally, the hybrid NMS split between FPGA and VEGA could be further tuned by profiling the actual latency of the pairwise suppression step on VEGA to determine the optimal value of K for the Top-K pre-filter stage.

VI. Conclusion

This work presents a hardware-software co-design for task-aware object detection on the Genesys-2 FPGA, built around the idea that unnecessary computation should be eliminated before it ever reaches the arithmetic units. By combining a quantized YOLOv8n backbone with a precomputed MiniLM-L6-v2 embedding lookup and a custom sparse CNN accelerator on the Kintex-7 fabric, the system handles task understanding, object detection, and edge constraints in a unified pipeline. Since the contest defines exactly 14 tasks, all embeddings are stored as a simple $O(1)$ BRAM lookup, removing any need for a language model at inference time and keeping VEGA free for preprocessing and NMS. Layered sparsity across task masking, structured 2:4 weight pruning, and scheduler-driven compute skipping means the DSP slices stay occupied only with meaningful work, which is a fundamentally different philosophy from prior accelerators that simply try to run dense operations faster.

The key open challenge going into Stage 2 is validating that QAT accuracy holds up under simultaneous 2:4 sparsity and INT8 quantization on the COCO-Tasks benchmark specifically, since task-relevant object selection is a stricter criterion than standard mAP. The hybrid INT8/INT16 detection head and the Top-K NMS split between FPGA and VEGA will also need empirical tuning once the pipeline runs on real hardware. Both are concrete and tractable problems, and the architecture as proposed gives a strong foundation to tackle them in the stages ahead.

VII. References

1. FINN: Dataflow compiler for QNN inference on FPGAs. <https://xilinx.github.io/finn/>
2. Efficient Layer-Wise N:M Sparse CNN Accelerator with Flexible SPEC: Sparse Processing Element Clusters - PMC. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10057003/>
3. A Sliding Window for Data Reuse in Deep Convolution Operations to Reduce Bandwidth Requirements and Resource Utilization. <https://www.mdpi.com/2079-9292/14/3/582>
4. Integration of a systolic array based hardware accelerator into a DNN operator auto-tuning framework. <https://arxiv.org/pdf/2212.03034>
5. Mobile-X: Dedicated FPGA Implementation of the MobileNet Accelerator Optimizing Depthwise Separable Convolution. https://www.researchgate.net/publication/382958443_Mobile-X_Dedicated_FPGA_Implementation_of_the_MobileNet_Accelerator_Optimizing_Depthwise_Separable_Convolution
6. Genesys 2 Kintex-7 FPGA Development Board. https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/2276/410-300_Web.pdf
7. An optimized YOLOv8n approach and its application to object detection problems - Francis Academic Press, accessed on March 18, 2026. <https://francispress.com/uploads/papers/egYUBaLXcLmYAzZDKRAyxocegjB1f01JBRUmKWVt.pdf>
8. Architecture Design of a Convolutional Neural Network Accelerator for Heterogeneous Computing Based on a Fused Systolic Array - PMC, accessed on March 18, 2026. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12845848/>
9. Optimized Implementation of YOLOv3-Tiny for Real-Time Image and Video Recognition on FPGA - MDPI, accessed on March 18, 2026. <https://www.mdpi.com/2079-9292/14/20/3993>
10. A Low-Power General Matrix Multiplication Accelerator with Sparse Weight-and-Output Stationary Dataflow - MDPI, accessed on March 18, 2026. <https://www.mdpi.com/2072-666X/16/1/101>
11. Performance Optimization of FPGA-Accelerated YOLOv8 for Driver Drowsiness Detection Using Vivado HLS - ResearchGate, accessed on March 18, 2026. https://www.researchgate.net/publication/392363887_Performance_Optimization_of_FPGA-Accelerated_YOLOv8_for_Driver_Drowsiness_Detection_Using_Vivado_HLS
12. Efficient Layer-Wise N:M Sparse CNN Accelerator with Flexible SPEC: Sparse Processing Element Clusters - MDPI, accessed on March 18, 2026. <https://www.mdpi.com/2072-666X/14/3/528>
13. Efficient Layer-Wise N:M Sparse CNN Accelerator with Flexible ..., accessed on March 18, 2026. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10057003/>
14. An FPGA-Based YOLOv5 Accelerator for Real-Time Industrial Vision Applications - PMC, accessed on March 18, 2026. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11434529/>
15. Double MAC on a DSP: Boosting the Performance of Convolutional Neural Networks on FPGAs - Krest Technology, accessed on March 18, 2026. <http://kresttechnology.com/krest-academic-projects/krest-mtech-projects/ECE/M-TECH%20VLSI%202019-20/BASE%20PAPERS/17.pdf>

Design Contest Stage 1 Report

16. Double MAC on a DSP: Boosting the Performance of Convolutional Neural Networks on FPGAs | Request PDF - ResearchGate, accessed on March 18, 2026.
https://www.researchgate.net/publication/324265271_Double_MAC_on_a_DSP_Boosting_the_Performance_of_Convolutional_Neural_Networks_on_FPGAs
17. A Survey on FPGA-Based Object Detection Using Various YOLO Techniques - IRJET, accessed on March 18, 2026. <https://www.irjet.net/archives/V12/i7/IRJET-V12I7103.pdf>
18. FPGA-Based CNN Acceleration on Zynq-7020 for Embedded Ship Recognition in Unmanned Surface Vehicles - PMC, accessed on March 18, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12986546/>
19. Software Development for 64-Bit RISC-V Processor Verification - Sched, accessed on March 18, 2026.
https://static.sched.com/hosted_files/riscvglobalforum2020/5c/RISC-V_GlobalForum_cd_ac_sreenadh.pdf
20. T. Mishra et al., "Accelerating Sparse Deep Neural Networks," *NVIDIA Technical Report*, 2021. <https://arxiv.org/abs/2104.08378>
21. [R16] Y. Li et al., "Gated Graph Sequence Neural Networks," *ICLR*, 2016.
<https://arxiv.org/abs/1511.05493>
22. T. Chen et al., "DGNN-Booster: A Generic FPGA Accelerator Framework For Dynamic Graph Neural Network Inference," *IPDPS*, 2023.
23. W. Wang et al., "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," *NeurIPS*, 2020.
<https://arxiv.org/abs/2002.10957>
24. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *EMNLP*, 2019. <https://arxiv.org/abs/1908.10084>
25. M. Courbariaux et al., "Training Neural Networks with Low-Precision Weights and Activations," *arXiv:1412.7024*, 2014.
26. Pappalardo et al., "Brevitas: Neural Network Quantization in PyTorch," *ICML MLOSS Workshop*, 2021. <https://github.com/Xilinx/brevitas>